

2024春 · 数据结构期末模拟赛

Author : Liop, Kai_Ker, Xhesica, AOSTL

单选题

1

下列排序方法中，稳定的排序方法是_C_。

A. 选择排序 B. 快速排序 C. 冒泡排序 D. 堆排序

解析：稳定的排序有插入排序、冒泡排序、归并排序、基数排序等，不包括选择、快速和堆排序

2

若在有序序列中采用折半查找方法进行查找，用来描述该查找过程的“判定树”的形状与_D_有关。

A. 序列中元素的值
B. 序列中元素的排列次序
C. 序列中元素的类型
D. 序列中元素的个数

解析：判定树是一种“完全”的树，用来描述决策过程，节点是每一次决策，则这棵树的形状与具体决策时的元素无关，仅仅与折半查找这种方法有关，所以所有折半查找的判定树形状都是相似的，唯一不同的只有二分查找到一个具体元素所需要的次数不同导致的判定树深度不同，所以只有序列中元素的个数会影响判定树的形状

3

有一无向图 $G=(V,E)$,其中: $V=\{a,b,c,d,e,f\}$, $E=\{(a,b),(a,e),(a,c),(b,d),(c,f),(f,d),(e,c)\}$, 则下面的顶点序列中_D_是该图深度优先遍历的一个正确的输出序列。

A. a,b,e,c,d,f
B. a,c,f,e,b,d

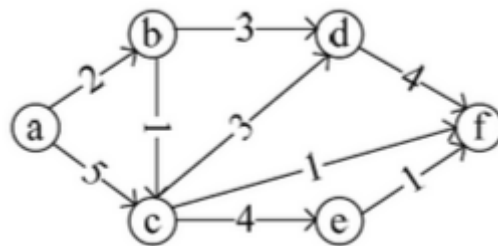
C. a,e,b,f,c,b

D. a,e,c,f,d,b

解析：深度优先遍历其过程简要来说是对每一个可能的分支路径深入到不能再深入为止，且每个节点只访问一次，则根据深度优先遍历的原则，仅有D选项满足

4

一有向带权图如下图所示，若采用迪杰斯特拉（Dijkstra）算法求源点a到其他各顶点的最短路径，得到的各最短路径的目标顶点顺序是_D_。



A. b, c, d, e, f

B. b, c, f, e, d

C. c, e, b, d, f

D. b, c, f, d, e

解析：迪杰斯特拉算法主要特点是从起始点开始，采用贪心算法的策略，每次遍历到始点距离最近且未访问过的顶点的邻接节点，直到扩展到终点为止。下面模拟Dijkstra算法的过程来展示选项正确性

第 0 步访问过的节点与距离：

a: 0

第 1 步访问过的节点与距离：

a: 0

b: 2

第 2 步访问过的节点与距离：

a: 0

b: 2

c: 3

第 3 步访问过的节点与距离:

a:0

b:2

c:3

f:4

第 4 步访问过的节点与距离:

a:0

b:2

c:3

f:4

d:5

第 5 步访问过的节点与距离:

a:0

b:2

c:3

f:4

d:5

e:7

5

下列说法中，错误的是 A。

- A. 具有 n 个点的无向连通图的邻接矩阵至少有 $2n-1$ 个非零元素
- B. 若有向图采用邻接表的存储方式，则其第 i 个链表中边结点个数是第 i 个顶点的出度。
- C. 对于给定的带权连通无向图，从某源点到图中各顶点的最短路径构成的生成树可能是该图的最小生成树。
- D. 包含具有 n 个顶点的连通图 G 的全部 n 个顶点,仅包含其 $n-1$ 条边的极小连通子图称为 G 的一个生成树。

解析: n 个点的无向连通图最少有 $n - 1$ 条无向边，则在邻接矩阵里面最少有 $2n - 2$ 个非零元素

6

以下删除链表中 p 指针所指结点的下一个结点的语句正确的是 D。

- A. `free(p→link); p→link = NULL;`
- B. `p→link = p→link→link; free(p→link);`

C. `free(p→link); p→link = p→link→link;`

D. `q = p→link; p→link = q→link; free(q);`

解析：

A选项释放了p指向的下一个节点的内存，但是没有维护链表的结构

B选项首先维护了链表的结构，但是在维护后p->link指向的已经是p的下下一个节点了，后续释放p->link的内存破坏了链表的结构

C选项首先释放了p指向的下一个节点的内存，导致记录p的下下个节点地址的信息丢失，p->link此时已经没有指向任何有意义的地址，后续赋值语句会导致内存错误

D选项首先保存了p的下个节点的地址，然后维护了链表的结构，再进行内存的释放，是正确的

7

栈和队分别是什么类型的数据结构？ **B**

A. 先进先出 先进后出

B. 先进后出 先进先出

C. 后进先出 随机读取

D. 随机读取 后进后出

解析： 本题考查栈和队列的定义

8

若五个元素的出栈序列为 1,2,3,4,5，则下列出栈序列不可能是该栈的入栈序列的是 **C**。

A. 1,2,3,4,5

B. 3,2,1,5,4

C. 2,3,1,4,5

D. 1,2,4,3,5

解析： 对于选项C，1入栈的时候2，3必须已经入栈且仍在栈中，则2无法先于3出栈，C错误

9

若有以下说明和语句:

```
struct elevator{
    int id;
    int floor;
    struct {
        char name[20];
        char tar;
    } passenger;
};
struct elevator elevators[2] = {{1, 2, {"Red", 'q'}}, {4, 5, {"X_ele", 'e'}}};
struct elevator *p = elevators;
```

则以下对语句与输出不符的是: **C**

- A. `printf("%d\n", (p+1)→id);`, 输出 4
- B. `printf("%d\n", strlen(p→passenger.name));`, 输出 3
- C. `printf("%c\n", *((*p).passenger.name+2));`, 输出 e
- D. `printf("%c\n", (*(p+1)).passenger.tar);`, 输出 e

解析: `*((*p).passenger.name+2)` 也即 `elevators[0].passenger.name[2]`, C 选项语句输出结果应为 d

10

设 p 指针指向单链表 (单链表长度为 n) 中的某个结点 ($p \neq \text{NULL}$), 若只知道指向该单链表第一个结点的指针和 p 指针, 则在 p 指针所指结点之前和 p 指针所指结点之后插入一个结点的时间复杂度分别是: **B**

- A. $O(1)$ 和 $O(n)$
- B. $O(n)$ 和 $O(1)$
- C. $O(n)$ 和 $O(n)$
- D. $O(1)$ 和 $O(1)$

解析: 对于单链表, 查找指定元素的前一个结点需要从头结点开始遍历链表, 复杂度为 $O(n)$, 而在指定结点后插入一个结点的复杂度为 $O(1)$, 所以 B 选项正确。

填空题

11

设栈S和队列Q的初始状态为空，元素e1、e2、e3、e4、e5、e6、e7依次进栈S，一个元素出栈后即进队列Q，若6个元素出队的序列是e2、e3、e6、e7、e5、e4，e1，则栈S的容量与队列Q的容量之和至少应该是_5_。

解析：对于单个队列的任何调整元素入队出队的顺序的操作，都无法改变最终出队的顺序，所以队列只需要有 1 个元素的容量即可。若我们想要让两个元素出栈顺序和入栈顺序相反，则需要将先入栈的元素保存在栈中，所以本题中栈的容量需要大于等于最长的元素编号递减的元素列长度，本题中是e7、e5、e4，e1，共 4 个元素，所以最终容量之和为 5

12

设有一组记录的关键字为{6, 42, 27, 19, 28, 17, 43, 10, 45, 26, 31, 21, 39, 9}，用用链地址法构造散列列表，散列函数为 $H(key)=key \text{ MOD } 13$ ，则散列地址为 6 的链中有_3_个结点。

解析：散列地址 i 中结点个数就是导致散列函数碰撞（即散列函数结果为 i ）的元素个数，本题中6，19，45这三个元素会发生碰撞，所以答案为 3

13

已知二叉树的前序序列为：ABCDEFGHJIJ，中序序列为：ACEDFBHJG，则其后序序列为_EFDCIJHGBA_。

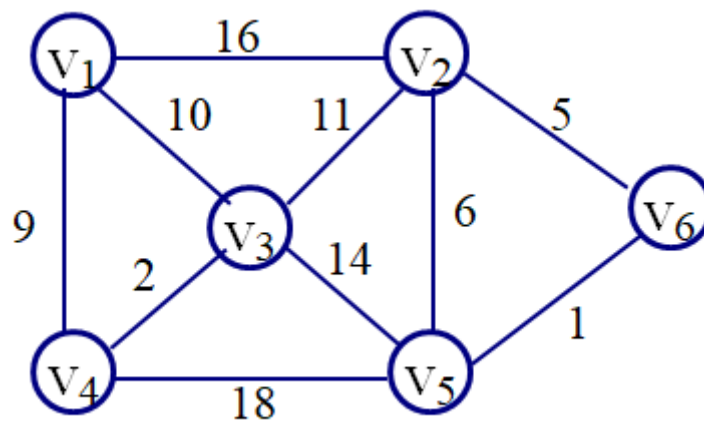
解析：本题考查通过前序序列和中序序列获得二叉树与后序序列。核心过程如下：首先通过前序序列确定当前根节点，在中序序列中找到根节点所在的位置，根节点之前的元素作为其左子树，根节点之后的元素作为其右子树，然后对于其左子树和右子树分别重复这个过程，直到所有元素都在二叉树中。

14

若用一个大小为20的数组（下标从0开始）来实现循环队列，且当前rear和front的值分别为18和10，当从队列中出队两个元素，再入队五个元素后，rear和front的值分别为_3_12_。

解析：循环队列的尾指针到达数组末尾的时候会返回数组开头，所以由题意rear和front的值分别为 $(18 + 5) \% 20 = 3$ 和 $(10 + 2) \% 20 = 12$ （注意提问顺序！）

15



对于上图所示的无向连通图，若采用普里姆（Prim）算法求其最小生成树，假设第一个选择加入最小生成树的顶点为**V2**，则最后一条加入最小生成树的边的权值为**9**。

解析：根据Prim算法的定义，这里加入边的顺序为5, 1, 11, 2, 9，所以答案为 9

16

表达式 $a+b/(c-d)-e*f$ 的后缀表达式是 **$abcd-/+ef*-$** 。

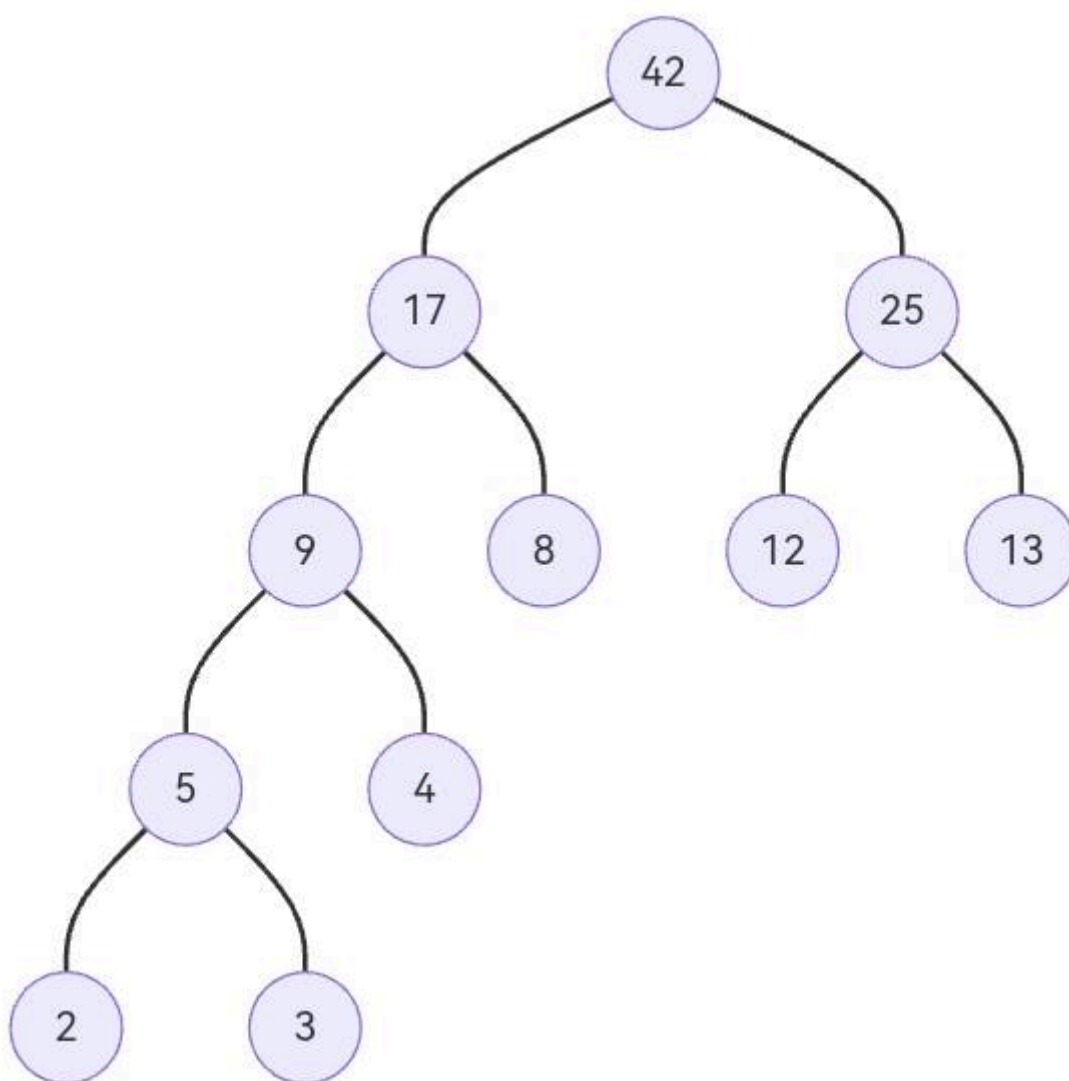
解析：本题考查由中缀表达式获得表达式树和后缀表达式。模拟表达式计算时的符号栈和数字栈的原理，生成表达式树。核心思路如下：

每次符号栈弹出符号进行运算的时候，将这个符号作为父节点，分别连接从数字栈弹出的两个数字所代表的表达式树的根节点，得到的运算结果代表这棵新的表达式树，最后将这个运算结果加入数字栈中，反复重复这个过程直到符号栈中没有符号为止

17

若以 {2, 3, 4, 8, 12, 13} 作为叶子结点的权值构造哈夫曼树，则其带权路径长度是 **98**。

解析：本题考查哈夫曼树的构造，题目确定的哈夫曼树的带权路径长度为
 $2 * 4 + 3 * 4 + 4 * 3 + 8 * 2 + 12 * 2 + 13 * 2 = 98$ 。哈夫曼树如下：



18

若一颗完全二叉树最深层有 14 个叶子节点，则其至少有_0_个度为 1 的节点。

解析：由完全二叉树的定义，完全二叉树度为1的节点只可能有0个或1个，当叶子节点数为偶数的时候没有度为1的节点，为奇数的时候有1个，所以本题答案为 0

19

假设采用快速排序算法按照从小到大的顺序对 10 个数据元素进行排序，这 10 个数据的初始状态为 (44, 32, 19, 31, 47, 26, 34, 48, 30, 26)，若第一次选用第一个数据 44 作为分界元素，则第一趟排序后，分界元素 44 前有_7_个数据元素。

解析：由快速排序的原理，每一趟排序都把比分界元素小的放到分界元素一侧，比分界元素大的放到另一侧。由题目条件，共有 7 个元素比分界元素小，所以答案为 7

20

如果输入的序列是 e1,e2,e3,e4,e5,e6,e7,e8,e9，如果使用多个队列，使其出队的序列为 e8,e4,e2,e5,e3,e9,e1,e6,e7，则至少需要_4_个这样的队列。（要求：输入的元素可以选择进入任何一个队列，但是每个队列都需要遵循先入先出的原则。）

解析：由于队列不会改变元素之间的顺序，所以编号大的元素若要在编号小的元素之前，则这两个元素不能在同一队列中，由此规则，e8,e4,e2,e1分别作为不同队列的队首元素，共需要四个队列

栈帧

```
#include <stdio.h>
#include <string.h>

/// 存储函数名以及栈空间大小
typedef struct {
    char name[32];
    int size;
} func_t;

/// 刻画栈中当前状态
typedef struct {
    func_t funcs[100];
    /// 栈中函数个数
    int size;
    /// 当前栈空间大小
    int record;
} record_t;

record_t store, stack, longest, largest;

int main() {
    char command[32];

    scanf("%d", &store.size);
    for (int i = 0; i < store.size; ++i) {
        scanf("%s%d", store.funcs[i].name, &store.funcs[i].size);
    }

    do {
        scanf("%s", command);
```

```

    if (strcmp(command, "call") == 0) {
        func_t func;
        scanf("%s", func.name);
        // 遍历查找，找到当前输入函数的栈空间大小
        for (int i = 0; i < store.size; i++) {
            if (strcmp(func.name, store.funcs[i].name) == 0) {
                func.size = store.funcs[i].size;
                break;
            }
        }
        // 将函数入栈
        stack.funcs[stack.size++] = func;
        // 更新栈空间
        stack.record += func.size;
        // 更新最多的以及最大的
        if (stack.size > longest.size) {
            longest = stack; // 注意！这里使用了结构体变量直接赋值，进行了内存的复制
        }
        if (stack.record > largest.record) {
            largest = stack;
        }
    } else if (strcmp(command, "return") == 0) {
        // 函数返回，简单处理即可
        stack.size--;
        stack.record -= stack.funcs[stack.size - 1].size;
    } else if (strcmp(command, "frame") == 0) {
        // 注意理解题目所说的含义，实质上是将栈中的函数保留到 x + 1 个
        scanf("%d", &stack.size);
        stack.size++;
        // 重新计算栈空间大小即可，比较省事
        stack.record = 0;
        for (int i = 0; i < stack.size; ++i) {
            stack.record += stack.funcs[i].size;
        }
    }
} while (stack.size != 0); // 注意循环条件

printf("%d\n", longest.size);
for (int i = 0; i < longest.size; i++) {
    printf("%s(%d) ", longest.funcs[i].name, longest.funcs[i].size);
}

printf("\n%d\n", largest.record);
for (int i = 0; i < largest.size; i++) {
    printf("%s(%d) ", largest.funcs[i].name, largest.funcs[i].size);
}
return 0;
}

```

东二食堂模拟

本题目的难点在于如何实现一个双端队列，支持在队头插入。
可以考虑使用循环队列实现。
其余部分按照题意模拟操作即可。

```
#include<stdio.h>
#include<string.h>
#include<math.h>
#include<stdlib.h>
struct rec{
    int x;int y;int z;
    int i;int k;
}re[105];
int cmp_x(const void *a, const void *b) {
    const struct rec *pa = (const struct rec *)a;
    const struct rec *pb = (const struct rec *)b;
    if(pa->x<pb->x)
        return -1;
    if(pa->x==pb->x)
        return 0;
    if(pa->x>pb->x)
        return 1;
}
int cmp_y(const void *a, const void *b) {
    const struct rec *pa = (const struct rec *)a;
    const struct rec *pb = (const struct rec *)b;
    if(pa->y<pb->y)
        return -1;
    if(pa->y==pb->y)
        return 0;
    if(pa->y>pb->y)
        return 1;
}
int queue[1005];
int mod=1005;
int rear=-1;int front=0;
int size(){
    return front-rear-1;
}
int empty(){
    return size()==0;
}
void push_back(int v){
    queue[((rear--)%mod+mod)%mod]=v;
}
void push_front(int v){
    queue[((front++)%mod+mod)%mod]=v;
}
```

```

int pop_front(){
    return queue[(((--front)%mod+mod)%mod)];
}
int main(){
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; ++i) {
        scanf("%d %d %d", &re[i].x, &re[i].y, &re[i].z);
        re[i].i = i;
    }
    qsort(re, n, sizeof(struct rec), cmp_x);
    int max_cycle = re[n - 1].x; //纪录最后一个人到达的时间
    int cycle = 1, cur_i = 0;
    //时间范围很小，可以直接按照时间进行模拟
    while (cycle <= max_cycle || !empty()) { //队列不为空或者最后一个人没来就进行
        循环
        while (cur_i < n && re[cur_i].x == cycle) { //尝试让一个人入队
            if (size() < re[cur_i].z) {
                push_front(cur_i);
            } else {
                push_back(cur_i);
            }
            cur_i++;
        }
        if (!empty()) { //尝试让队首出队
            int i = pop_front();
            re[i].k = cycle;
        }
        cycle++;
    }
    qsort(re, n, sizeof(struct rec), cmp_y);
    for (int i = 0; i < n; ++i) {
        printf("%d %d\n", re[i].y, re[i].k);
    } //输出结果
    return 0;
}

```

JML

题意描述

- n 个点的树，要求 $O(1)$ 实现查询父亲、增加儿子、改变父亲， $O(n)$ 实现改变所有儿子的父亲
- 对于一个不定长的队列， $O(1)$ 实现查询和末尾增加，不能直接开数组

题解

对于第一个问题，我们采用邻接链表实现树的存储，同时对于每个节点记录其父亲节点。

用一个数组保存所有节点，下标代表节点 id，实现 $O(1)$ 的 id-节点 索引。

- 查询父亲：索引对应的节点，查询记录的父亲即可
- 增加儿子：初始化节点即可
- 改变父亲：取出对应节点，修改节点的父亲记录，然后把对应节点插入邻接链表即可
- 改变所有儿子的父亲：取出节点，邻接链表遍历儿子，修改儿子的父亲，然后加入对应的邻接链表

这样可以得到 40 分。

对于第二个问题，我们在每个节点都只用数组存储几个指针（比如 N 个）。这 N 个指针的作用是
指向 N 个数组，每个数组可以是 $SIZE$ 个 `int` 的大小。

那么对于一个节点的队列而言，队列的第 i 个元素（从 0 开始）是第 $i / SIZE$ 个指针指向的数组的
第 $i \% SIZE$ 个元素。这样查询和增加的过程都可以通过数组下标的索引完成，时间复杂度是
 $O(1)$ 。

需要注意的是，这些指针指向的数组并不在（也不能在）编译时初始化，而是在运行时按需申请：
假设当前已经申请了 i 个数组（初始化了从 1 开始的第 i 个指针），那么当前的队列容量为 $i * SIZE$ ，
如果需求超过了这个容量，就申请第 $i + 1$ 个数组，使队列的容量扩大到 $(i + 1) * SIZE$ 。

本题考察树的基本应用和 `malloc()` 的使用。

测试点说明

第一个测试点：检查是否 $O(n)$ 实现改变所有儿子的父亲和 $O(1)$ 实现其他与树有关的需求

第二个测试点：检查队列是否能够正确处理大规模数据

第三个测试点：检查每个节点的队列是否都可以处理数据

第四个测试点：检查是否和树有关的指令的正确性

第五个测试点：检查队列实现正确性

第六个测试点：综合测试

第七个测试点：检查是否 $O(1)$ 实现改变所有儿子的父亲，挑战性任务不占分。提示：并查集。

其他测试注意事项：`message` 的长度可能为 0，不能用 `scanf` 直接读入；入职的员工不能继承离职员工的消息。

标程

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Person {
    int fa_ptr;
    int self_ptr;

    int* messages[15];
    int messages_capacity;
    int messages_size;
} person;
person persons[300005];
typedef struct Message {
    int id;
    int content;
    int sender;
} message;
char message_content[100005][101];
int cnt_message;
message messages[100005];

int n, m;
int pointer[300005], cnt_ptr, is_end[300005];
const char* COMMANDS[] = {"queryMessage", "queryLeader", "sendMessage",
    "depart", "enter", "change"};

void add_person(int u, int v) {
    if (persons[u].self_ptr == 0) {
        pointer[persons[u].self_ptr = ++cnt_ptr] = u;
        is_end[cnt_ptr] = 1;
    }
    persons[v].fa_ptr = persons[u].self_ptr;
    persons[v].messages[0] = (int*)malloc(sizeof(int) * 16);
    persons[v].messages_capacity = 4;
    persons[v].messages_size = 0;
}

void queryMessage(int id, int ith) {
    if (id != 1 && persons[id].fa_ptr == 0) {
        puts("-1");
        return;
    } else if (ith > persons[id].messages_size) {
        puts("-1");
        return;
    }
    for (int i = 4, j = 0; i <= persons[id].messages_capacity; i++) {
        if (ith <= (1 << i)) {
```

```

        int message_id = persons[id].messages[i - 4][ith - j - 1];
        printf("%d %s %d\n", message_id,
message_content[messages[message_id].content], messages[message_id].sender);
        return;
    } j = 1 << i;
}
}

void addMessage(int id, int sender) {
    messages[cnt_message].id = cnt_message;
    messages[cnt_message].content = cnt_message;
    messages[cnt_message].sender = sender;
    if (persons[id].messages_size == (1 << persons[id].messages_capacity)) {
        persons[id].messages[persons[id].messages_capacity - 3] =
(int*)malloc(sizeof(int) * (1 << persons[id].messages_capacity));
        persons[id].messages_capacity++;
    } persons[id].messages_size++;
    int delta;
    if (persons[id].messages_capacity == 4) delta = 0;
    else delta = 1 << (persons[id].messages_capacity - 1);
    persons[id].messages[persons[id].messages_capacity - 4]
[persons[id].messages_size - delta - 1] = cnt_message;
    puts("0");
}

int find(int ptr) {
    if (is_end[ptr]) return ptr;
    else return pointer[ptr] = find(pointer[ptr]);
}

void queryLeader(int id) {
    if (id == 1 || persons[id].fa_ptr == 0) {
        puts("-1");
        return;
    } printf("%d\n", pointer[find(persons[id].fa_ptr)]);
}

void depart(int id1, int id2) {
    if (id1 != 1 && persons[id1].fa_ptr == 0) {
        puts("-1");
        return;
    } else if (id2 != 1 && persons[id2].fa_ptr == 0) {
        puts("-1");
        return;
    }
    if (persons[id2].self_ptr == 0) {
        persons[id2].self_ptr = ++cnt_ptr;
        pointer[cnt_ptr] = id2;
        is_end[cnt_ptr] = 1;
    }
}

```

```

        if (persons[id1].self_ptr) {
            pointer[persons[id1].self_ptr] = persons[id2].self_ptr;
            is_end[persons[id1].self_ptr] = 0;
            persons[id1].self_ptr = 0;
        }
        persons[id1].fa_ptr = 0;
        puts("0");
    }

void enter(int id1, int id2) {
    if (id1 == 1 || persons[id1].fa_ptr != 0) {
        puts("-1");
        return;
    } else if (id2 != 1 && persons[id2].fa_ptr == 0) {
        puts("-1");
        return;
    }
    if (persons[id2].self_ptr == 0) {
        persons[id2].self_ptr = ++cnt_ptr;
        pointer[cnt_ptr] = id2;
        is_end[cnt_ptr] = 1;
    }
    persons[id1].fa_ptr = persons[id2].self_ptr;
    persons[id1].messages_capacity = 4;
    persons[id1].messages_size = 0;
    if (persons[id1].messages[0] == NULL)
        persons[id1].messages[0] = (int*)malloc(sizeof(int) * 16);
    puts("0");
}

void change(int id1, int id2) {
    if (id1 != 1 && persons[id1].fa_ptr == 0) {
        puts("-1");
        return;
    } else if (id2 != 1 && persons[id2].fa_ptr == 0) {
        puts("-1");
        return;
    }
    if (persons[id2].self_ptr == 0) {
        persons[id2].self_ptr = ++cnt_ptr;
        pointer[cnt_ptr] = id2;
        is_end[cnt_ptr] = 1;
    }
    persons[id1].fa_ptr = persons[id2].self_ptr;
    puts("0");
}

void handout(char* command) {
    int op;
    int len = strlen(command);

```



```

for (op=0;op<6;op++)
    if (strstr(command, COMMANDS[op]) == command)
        break;
if (op == 0) {
    int i;
    for (i = 0; i < len; i++)
        if (command[i] == ' ')
            break;
    int id = 0, ith = 0;
    for (int j = i + 1; j < len; j++)
        if (command[j] == ' ') {
            for (i++; i < j; i++)
                id = id * 10 + command[i] - '0';
            i = j;
            break;
        }
    for (i++; i < len; i++)
        ith = ith * 10 + command[i] - '0';
    queryMessage(id, ith);
} else if (op == 1) {
    int id = 0;
    int i;
    for (i = 0; i < len; i++)
        if (command[i] == ' ')
            break;
    for (i++; i < len; i++)
        id = id * 10 + command[i] - '0';
    queryLeader(id);
} else if (op == 2) {
    int i;
    for (i = 0; i < len; i++)
        if (command[i] == ' ')
            break;
    int id1 = 0, id2 = 0;
    for (int j = i + 1; j < len; j++)
        if (command[j] == ' ') {
            for (i++; i < j; i++)
                id1 = id1 * 10 + command[i] - '0';
            i = j;
            break;
        }
    for (int j = i + 1; j < len; j++)
        if (command[j] == ' ') {
            for (i++; i < j; i++)
                id2 = id2 * 10 + command[i] - '0';
            i = j + 1;
            break;
        }
    if (id1 != 1 && persons[id1].fa_ptr == 0) {
        puts("-1");
    }
}

```

```

        return;
    } else if (id2 != 1 && persons[id2].fa_ptr == 0) {
        puts("-1");
        return;
    } cnt_message++;
    for (int k = 0; k < len; k++)
        message_content[cnt_message][k] = command[k];
    addMessage(id2, id1);
} else if (op == 3 || op == 4 || op == 5) {
    int i;
    for (i = 0; i < len; i++)
        if (command[i] == ' ')
            break;
    int id1 = 0, id2 = 0;
    for (int j = i + 1; j < len; j++)
        if (command[j] == ' ') {
            for (i++; i < j; i++)
                id1 = id1 * 10 + command[i] - '0';
            i = j;
            break;
        }
    for (i++; i < len; i++)
        id2 = id2 * 10 + command[i] - '0';
    if (op == 3) depart(id1, id2);
    else if (op == 4) enter(id1, id2);
    else change(id1, id2);
}
}

int main() {
    scanf ("%d",&n);
    persons[1].messages[0] = (int*)malloc(sizeof(int) * 16);
    persons[1].messages_capacity = 4;
    persons[1].messages_size = 0;
    for (int i=1;i<n;i++) {
        int u, v;
        scanf ("%d %d", &u, &v);
        add_person(u, v);
    } scanf ("%d",&m);getchar();
    for (int i=1;i<=m;i++) {
        char command[500];
        gets(command);
        handout(command);
    }
    return 0;
}

```

Fat Musha

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/// 错误类型
enum { E_NOT_FOUND = 1, E_TYPE_ERROR, E_NO_MEM };

/// 节点类型，分别表示：文件、文件夹、硬链接
typedef enum { REG, DIR, LNK } type_t;

/// 树的节点，采用链表维护不定子节点树
typedef struct __node_t {
    type_t type;
    char name[33];
    int limit; // 对于文件来说，表示大小；对于文件夹来说，表示容量限制
    struct __node_t *first_child; // 硬链接也用这个来表示
    struct __node_t *next_sibline;
} node_t;

/// 将路径按照 '/' 进行分割，每部分起始的指针依次放入 splitted 中
void split_path(char **splitted, char *path) {
    int count = 0;
    splitted[count++] = path;
    for (char *c = path; *c; ++c) {
        if (*c == '/') {
            *c = '\\0';
            splitted[count++] = c + 1;
        }
    }
    splitted[count] = NULL; // 结束标记
}

/// 在 proot 下查找子节点 name，返回 0 表示正确，否则为相应错误
int find_child(const char *name, node_t *proot, const type_t *ptype, node_t **out) {
    for (node_t *p = proot->first_child; p; p = p->next_sibline) {
        if (strcmp(p->name, name) == 0) {
            while (p->type == LNK) {
                p = p->first_child;
            }
            if (ptype) {
                // 如果需要检查类型，不需要检查的话 ptype == NULL
                if (p->type != *ptype) {
                    return -E_TYPE_ERROR;
                }
            }
            *out = p; // 将找到的节点存放在这里
        }
    }
}
```

```

        return 0;
    }
}
return -E_NOT_FOUND;
}

/// 根据完整的路径查找目标节点
int find_node(char **splited, node_t *proot, const type_t *ptype, node_t
**out) {
    node_t *node = proot;
    char **pname;
    // 从第一层路径开始遍历，直到倒数第二个
    for (pname = splited; pname[1]; pname++) {
        char *name = *pname;
        node_t *p;
        type_t dir_type = DIR; // 检验类型：应该是文件夹
        int ret = find_child(name, node, &dir_type, &p);
        if (ret != 0) {
            // 类型错了，直接返回
            return ret;
        }
        node = p;
    }
    // 最后一层路径，需要校验以下类型
    return find_child(*pname, node, ptype, out);
}

/// 遍历整棵树，检查每个节点的容量限制，返回当前节点的容量
int check(node_t *proot) {
    if (proot->type == LNK) {
        // 硬链接直接检查对应节点
        return check(proot->first_child);
    } else if (proot->type == REG) {
        // 文件直接返回自己的大小
        return proot->limit;
    }

    // 检查文件夹
    int limit = proot->limit;
    if (limit == 0) {
        // 没有限制
        limit = 0x7fffffff;
    }

    int sum = 0;
    // 遍历文件夹下的每一个文件
    for (node_t *p = proot->first_child; p; p = p->next_sibline) {
        int ret = check(p);
        if (ret == -E_NO_MEM) {
            // 遍历时，哪里发现容量不够了，就返回错误码

```

```

        return ret;
    }
    // 累加和
    sum += ret;
}

// 检查限制
if (sum > limit) {
    return -E_NO_MEM;
} else {
    return sum;
}
}

/// 创建文件夹，根据给定的路径
void mkdir(char **splited, node_t *proot) {
    node_t *node = proot;
    for (char **pname = splited; *pname; pname++) {
        char *name = *pname;
        node_t *p;
        int ret = find_child(name, node, NULL, &p);
        if (ret == -E_NOT_FOUND) {
            // 没找到这个文件/文件夹就创建
            p = node->first_child;
            node->first_child = (node_t *)malloc(sizeof(node_t));
            *node->first_child = (node_t){ .type = DIR, .first_child = NULL,
.next_sibling = p, .limit = 0 };
            strcpy(node->first_child->name, name);
            node = node->first_child;
        } else {
            node = p;
        }
    }
}

/// 限制容量
int limit(char **splited, node_t *proot, int lim) {
    node_t *p;
    (void)find_node(splited, proot, NULL, &p);
    int old = p->limit;
    p->limit = lim;
    if (check(proot) == -E_NO_MEM) {
        // 发现失败了，恢复对其的限制
        p->limit = old;
        return -1;
    }
    return 0;
}

/// 创建文件

```

```

int touch(char **splited, node_t *proot, type_t new_type, node_t *first_child,
node_t **pparent) {
    node_t *node = proot;
    char **pname;
    // 逐层找文件夹，当然跳过最后一层
    for (pname = splited; pname[1]; pname++) {
        char *name = *pname;
        node_t *p;
        type_t dir_type = DIR;
        int ret = find_child(name, node, &dir_type, &p);
        if (ret != 0) {
            return ret;
        }
        node = p;
    }
    if (node->name[0] == '/') {
        // 特判情况，放置写入 /qwert 这种文件
        return -1;
    }
    if (pparent) {
        // 返回父文件夹，如果需要的话
        *pparent = node;
    }
    node_t *p;
    int ret = find_child(*pname, node, NULL, &p);
    if (ret != -E_NOT_FOUND) {
        // 这里表示找到了，但是是不对的情况
        return -1;
    } else {
        // 头插，插入一个子节点
        p = node->first_child;
        node->first_child = (node_t *)malloc(sizeof(node_t));
        *node->first_child =
            (node_t){ .type = new_type, .first_child = first_child,
.next_sibling = p, .limit = 0 };
        strcpy(node->first_child->name, *pname);
        return 0;
    }
}

/// 创建硬链接
int mklnk(char **spl_dst, char **spl_src, node_t *proot) {
    node_t *node_src, *node_dst_parent;
    (void)find_node(spl_src, proot, NULL, &node_src);
    (void)touch(spl_dst, proot, LNK, node_src, &node_dst_parent);
    if (check(proot) == -E_NO_MEM) {
        // 撤销创建
        node_t *lnk = node_dst_parent->first_child;
        node_dst_parent->first_child = lnk->next_sibling;
        free(lnk);
    }
}

```

```

        return -1;
    }
    return 0;
}

void print(int ret) {
    if (ret == 0) {
        printf("YES\n");
    } else {
        printf("NO\n");
    }
}

int main() {
    // 题目要求的根目录
    node_t root = { .type = DIR, .name = { "root" }, .first_child = NULL,
.next_sibling = NULL };
    // 可以理解成头节点, 方便操作
    node_t base = { .type = DIR, .name = { "/" }, .first_child = &root,
.next_sibling = NULL };
    node_t *proot = &base;

    int T;
    scanf("%d", &T);

    char command[10], path1[400], path2[400];
    char *splited1[12], *splited2[12];
    int size;

    // 后面就是简单调一调函数, 非常爽的说
    while (T--) {
        scanf("%s", command);
        if (strcmp(command, "mkdir") == 0) {
            scanf("%s", path1);
            split_path(splited1, path1);
            mkdir(splited1, proot);
            printf("YES\n");
        } else if (strcmp(command, "limit") == 0) {
            scanf("%s%d", path1, &size);
            split_path(splited1, path1);
            print(limit(splited1, proot, size));
        } else if (strcmp(command, "touch") == 0) {
            scanf("%s", path1);
            split_path(splited1, path1);
            print(touch(splited1, proot, REG, NULL, NULL));
        } else if (strcmp(command, "edit") == 0) {
            scanf("%s%d", path1, &size);
            split_path(splited1, path1);
            print(limit(splited1, proot, size));
        } else if (strcmp(command, "mklnk") == 0) {

```

```
scanf("%5%S", path1, path2);
split_path(splited1, path1);
split_path(splited2, path2);
print(mklnk(splited1, splited2, proot));
    }
}
}
```